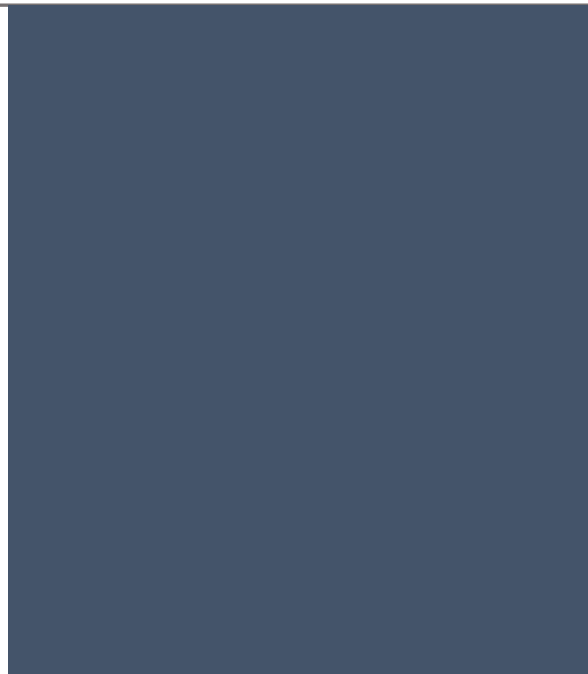




Projet Electronique

Implémentation d'une
machine virtuelle dans un
microcontrôleur

Yanis Boumghar et Antoine Choplin

Introduction

Ce projet électronique fait suite au projet informatique réalisé lors du second semestre. Après avoir réalisé un interpréteur de Grafcet en langage C nous souhaitons interpréter un grafcet sur microcontrôleur. Ce projet comporte plusieurs parties.

Premièrement, il va falloir intégrer la machine virtuelle réalisée en langage C dans le programme du microcontrôleur. Secondement, il va falloir adapter le programme afin qu'il soit adapté au système choisi.

Pour commencer, nous avons déterminé le scénario sur lequel on souhaite travailler : nous allons utiliser la maquette réalisée par les AGRALS. Il s'agit d'un tapis roulant composé de deux capteurs, chacun à une extrémité du tapis et un autre capteur qui servira de déclencheur pour le moteur qui fait avancer le tapis. On se servira des capteurs aux extrémités pour garder un « pièce » sur le tapis lorsqu'il arrivera au bout le moteur tournera dans le sens inverse et le tapis changera de sens.

Dans ce compte rendu, je vais vous présenter la réalisation de ce projet de la machine virtuelle implémenter sur le microcontrôleur au contrôle du moteur.

Table des matières

Introduction	1
1. La machine virtuelle.....	3
2. Contrôle Moteur	4
Conclusion.....	6

1. La machine virtuelle

La machine virtuelle réalisée en langage C représente une fonction dans le programme du microcontrôleur, la fonction `run()`. Elle va exécuter un tableau d'opcode, c'est-à-dire un tableau qui est composé d'instructions qui permet la manipulation d'une « pile » (la stack) où des calculs seront réalisés, par exemple des opérations logiques comme des AND, des OR ou encore des NOT. Ces instructions vont également interagir avec un tableau de variable il va venir lire et écrire des valeurs.

On va définir alors à l'avance où se situe les entrées et les sorties du système.

Variable[0]	Etat du bouton de la carte Nucleo
Variable[1]	Front montant du bouton de la carte Nucleo
Variable[2]	Front descendant du bouton de la carte Nucleo
Variable[3]	Etat du capteur start/off
Variable[4]	Front montant du capteur start/off
Variable[5]	Front descendant du capteur start/off
Variable[6]	Etat du capteur position1
Variable[7]	Front montant du capteur position1
Variable[8]	Front descendant du capteur position1
Variable[9]	Etat du capteur position2
Variable[10]	Front montant du capteur position2
Variable[11]	Front descendant du capteur position1
Variable[12]	LED0
Variable[13]	Moteur on/off
Variable[14]	Moteur Sense1
Variable[15]	Moteur Sense2

2. Réception de l'opcode par liaison série

Nous avons choisi de recevoir l'opcode capable de simuler le scénario souhaité à partir du fichier généré par l'assembleur réalisé en fichier C.

Pour cela nous avons côté PC un programme capable d'envoyer l'opcode. Afin de le recevoir nous utilisons des interruptions sur la liaison série USART2, lorsqu'un caractère est reçu nous le stockons en attendant que le caractère '@' qui confirme l'envoi de tous les caractères soit reçue. Lorsque c'est le cas on confirme cette réception en renvoyant au PC les mêmes caractères puis l'on vient placer l'opcode reçu dans le tableau correspondant.

C'est dans la fonction `USART2_IRQHandler()` que ce traitement est réalisée.

2. Contrôle Moteur

Afin de générer nos sorties il a fallu initialiser les pattes correspondantes aux sorties et aux entrées du système.

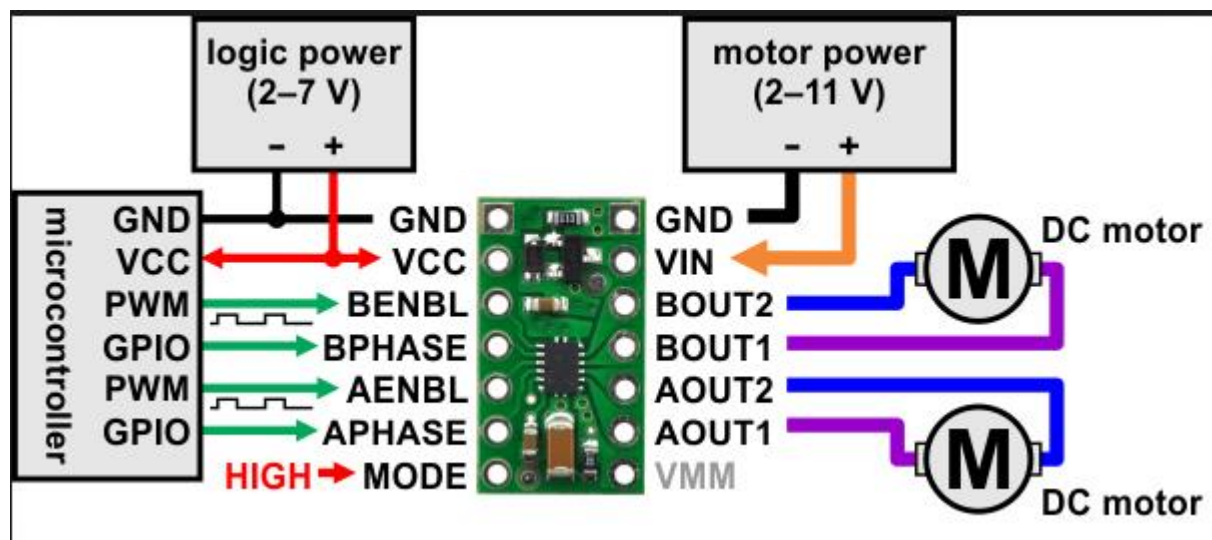
Nous avons configuré les pattes PC5, PC6, PC8 et PC13 comme des entrées et les pattes PA8 et PA12 comme des sorties.

Commençons par les entrées, les capteurs utilisés renvoient une information logique, 0 lorsqu'ils ne détectent rien et 1 lorsqu'ils détectent quelque chose.

Exemple d'initialisation en entrée :

```
//Initialisation de la patte PC13 en entrée (bouton)
void initPatte_PC13(void)
{
    RCC->AHBENR |=RCC_AHBENR_GPIOCEN;    // Activation horloge du GPIOB
    GPIOC->MODER &=~(1<<27);    // choix mode General Purpose Output bit b19 à '0'
    GPIOC->MODER &=~(1<<26);    // choix mode General Purpose Output bit b18 à '0'
    GPIOC->PUPDR &=~(1<<27);    // desactivation resistance de tirage: bit b11 à '0'
    GPIOC->PUPDR &=~(1<<26);    // desactivation resistance de tirage: bit b10 à '0'
}
```

Pour les sorties, nous avons dû nous renseigner sur l'information à envoyer pour obtenir le contrôle du moteur. Nous avons choisi d'utiliser un driver que l'on connaissait car nous l'avions déjà utilisé lors d'un autre projet. Le driver DRV8835.



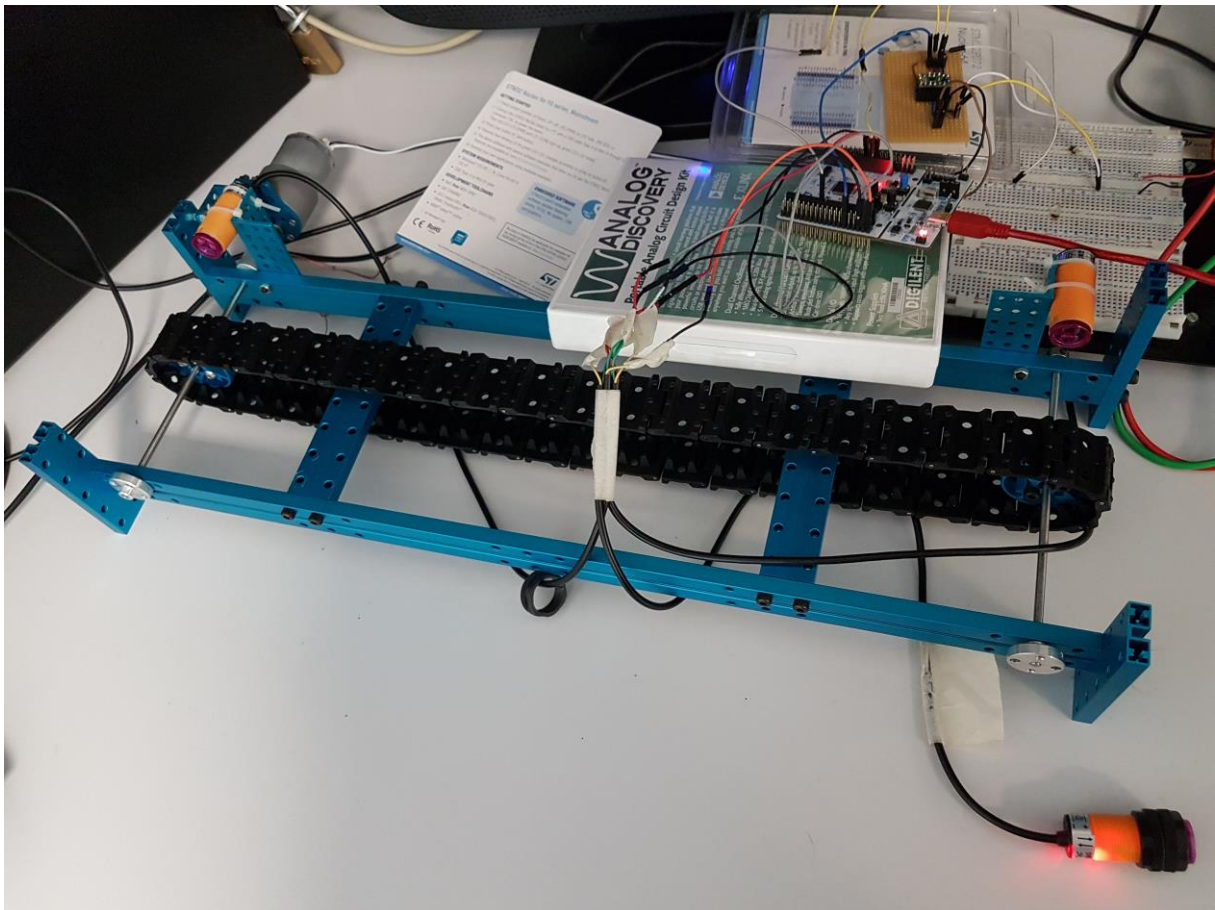
D'après sa documentation, en mode 1, il faut envoyer une PWM sur Benbl ou Aenbl et relier Bphase ou Aphase à la masse où à VCC pour obtenir un sens ou l'autre du moteur en sortie. Le moteur sera connecté entre Bout1 et Bout 2 ou entre Aout1 et Aout2.

Aphase/Bphase sera contrôlé par la machine virtuelle et fait référence au sens du moteur, elle prendra la valeur de la variable[14] ou variable [15] selon le sens du moteur qui est choisi.

Afin de faire le lien entre le microcontrôleur et le système nous avons réalisé un PCB capable d'accueillir ce driver.

Il ne nous reste donc plus qu'à générer une PWM sur Aenbl/Benbl relié à la patte pa8 du microcontrôleur. Nous avons choisi cette patte car il était possible d'y utiliser un timer. Nous avons utilisé le timer1.

Lorsque la valeur de moteur on/off sera à 0 le rapport cyclique sera de 0 et lorsque sa valeur sera 1 le rapport cyclique sera de 50%. C'est ce qui permettra d'allumer ou non le moteur.



Conclusion

Ce projet nous a permis de mettre différents apprentissages en commun afin d'obtenir un système complets comme un interpréteur de grafct de la compilation, côté software, jusqu'à son exécution sur un microcontrôleur, côté hardware. De plus travailler en partenariat avec une autre formation comme les AGRALs a été particulièrement motivant.

Après le projet des LEDs tournantes, ce deuxième projet nous a permis de conforter nos acquis en terme de microcontrôleur sur un nouveau système avec gestion d'entrées et de sorties en utilisant des fonctionnalités vu précédemment en cours comme les interruptions et les timers.